Sant'Anna School of Advanced Studies

The BioRobotics Institute

Course: Neuromorphic Engineering

Project Documentation and Review

Spike-Based Hand Gesture Recognition: A Comparative Study of Neuromorphic Encoding Strategies Using Izhikevich Neurons

Authors:

Andrea Diano - Filippo Aspi - Riccardo Mambrini

June 22, 2026

Contents

1 Introduction **2**
- 1.1 Leap Motion Controller . . . 2
  - 1.1.1 Hardware Limitations . . . 2
- 1.2 The Goal . . . 2
  - 1.2.1 Standard Spatial Geometry-Based Classification . . . 2
  - 1.2.2 Neuromorphic Classification via Spike Count Coding . . . 3
- 1.3 System Integration: LabVIEW and Python Pipeline . . . 3
- 1.4 Routing Mechanism . . . 4

2 LabVIEW **5**
- 2.1 Introduction & Usage . . . 5
  - 2.1.1 Acquisition . . . 5
  - 2.1.2 Data transformation . . . 5
  - 2.1.3 Sender and receiver . . . 5
- 2.2 Explanation of the Main VIs . . . 5

3 Python **7**
- 3.1 Inference . . . 7
- 3.2 Machine Learning . . . 7
  - 3.2.1 Model Training and Hyperparameter Optimization . . . 7

4 Results **8**
- 4.1 Neuromorphic Hand Gesture Classification . . . 8
- 4.2 Distance-Based Classification . . . 9

5 Temporal Window Optimization **11**
- 5.1 Experimental Results . . . 11
- 5.2 Accuracy vs. Latency Trade-off Analysis . . . 11

6 Standard vs. Neuromorphic Comparison **12**
- 6.1 Information Representation and Performance Trade-offs . . . 12
- 6.2 Real-time UDP Implementation: Theory vs. Practice . . . 12

7 Conclusions and Future Work **14**
- 7.1 Future Work . . . 14

1 Introduction

1.1 Leap Motion Controller

This project utilizes the Leap Motion Controller (LMC). This device is designed to track hand movements and extract features such as finger positions, hand rotation, and other related metrics. The LMC consists of two monochromatic IR cameras and three IR LEDs (emitters).

1.1.1 Hardware Limitations

It was impossible to establish a univocal channeling. When the LMC fails to detect one or more fingers, it typically decreases the size of the output data matrix without discriminating between the different fingers. To address this, we implemented a simple routing mechanism which is described in the section 1.4.

1.2 The Goal

This project aims to compare different methods that are able to classify six fundamental left hand gestures:

1. Open palm
2. Fist
3. Thumb only
4. Pinky only
5. Horns gesture
6. Thumb + pinky

We self-collected the dataset, recording each gesture for a duration of 1 minute and 30 seconds. During the recording, small spatial shifts were introduced to simulate varying conditions for the same gesture. We use only the left hand to reduce the complexity of the system.

To achieve this goal, the project evaluates and compares different data representation and encoding paradigms. Rather than relying on a single data structure, the pipeline processes the raw hand telemetry through two main families of features, each utilizing specific encoding strategies.

1.2.1 Standard Spatial Geometry-Based Classification

The first approach directly exploits the spatial features extracted from the Leap Motion tracking data. The dataset consists of the relative distances of the five fingers from the palm center, alongside the three-dimensional direction vectors of the palm itself.
In this pipeline, two feature configurations are evaluated and compared:

- **Distances Only (5 features)**: Utilizes only the 5 finger distances, providing a robust but geographically limited baseline.
- **Distances + Palm Direction (8 features)**: Enriches the finger distances by appending the three-dimensional direction vectors of the palm.

1.2.2 Neuromorphic Classification via Spike Count Coding

The second branch of the project explores the *Neuromorphic Computing*. Spatial time-series data are converted into spike trains through the mathematical modeling of spiking neurons using the **Izhikevich** model. This bio-inspired rate-coding maps the amplitude of physical telemetry onto the firing frequency of simulated neurons.

The analysis relies exclusively on the spike count within a fixed time window of 1 s. The inter-spike interval (ISI) was excluded from the framework, as it was deemed non-informative for the specific objectives of this study.

Four neuromorphic dataset configurations are analyzed to evaluate the impact of spatial data:

- **Blind Spike Encoding (5 features)**: Generates spike counts directly from raw spatial coordinates.
- **Summed Spike Encoding (1 feature)**: Aggregates the temporal activity by encoding the total sum of spikes across all the five channels, used in the Blind Spike encoding, into a single representative feature.
- **Routed Spike Encoding (5 features)**: Generates spike counts from coordinates routed to avoid the issue mentioned in 1.1.1.
- **BlindDir Spike Encoding (11 features)**: Combines raw finger spike counts with positive and negative palm direction vectors to preserve orientation context in the spike domain.

This structured comparison allows for a detailed analysis of how spatial normalization, orientation vectors, and bio-inspired rate coding affect classification performance.

1.3 System Integration: LabVIEW and Python Pipeline

The development and execution of the proposed system are divided between **LabVIEW** and **Python**, creating a hybrid pipeline where each environment handles specific tasks:

- **Real-Time Acquisition and Normalization (LabVIEW)**: LabVIEW serves as the interface for real-time hardware interaction with the Leap Motion sensor. It handles the acquisition of raw joint telemetry and coordinates. Crucially, the preprocessing and spatial tasks—such as the routing of finger coordinates, handling missing values, and the conversion in Izhikevich spiking neurons are executed directly within the LabVIEW environment.
- **Spiking Simulation, Training, and Optimization (Python)**: The preprocessed and normalized datasets are exported to Python, which performs the heavy computational tasks of the machine learning pipeline. Python is responsible for running the grid search tuning, and training the Random Forest models. Once trained, the optimized classifiers are serialized into portable model files (`.pkl`).
- **Closed-Loop Real-Time Deployment**: For real-time gesture prediction, the serialized Python models are loaded back into the LabVIEW execution runtime. LabVIEW acquires live hand positions, applies the real-time spatial normalization, and queries the Python models to output instantaneous gesture classifications.

1.4 Routing Mechanism

To achieve the finger routing, the spatial joint positions of the hand are decomposed into their individual x, y, and z coordinate axes. We use **only the x-axis coordinates** for the routing process.
The routing is centered on the palm of the hand. The relative coordinate for each finger i (where $i \in \{1, 2, 3, 4, 5\}$ corresponds to the thumb, index, middle, ring, and little finger, respectively) is calculated by subtracting the palm center's x-coordinate from the finger's x-coordinate:

$$x_{rel,i} = x_{finger,i} - x_{palm} \tag{1}$$

Since the dataset is recorded using the **left hand**, this relative routing onto a single lateral axis yields specific spatial properties:

- The **middle finger** (N_3) is aligned with the center of the palm along the x-axis, meaning its relative coordinate has a very small absolute value ($|x_{rel,3}| \approx 0$).
- The **ring finger** (N_4) and the **pinky/little finger** (N_5) lie to the left of the palm center, resulting in negative relative values ($x_{rel,4} < 0$ and $x_{rel,5} < 0$).
- The **index finger** (N_2) and the **thumb** (N_1) lie to the right of the palm center, resulting in positive relative values ($x_{rel,2} > 0$ and $x_{rel,1} > 0$).

To identify the specific finger, we employ fixed thresholds (found through trial and error experiments) applied to the amplitude of the relative distance along the x-axis ($|x_{rel,i}|$). Because the physical anatomy of the hand maintains a predictable relative sequence and spacing between fingers, these fixed thresholds partition the 1D lateral space into distinct intervals. This partitioning allows the model to map each coordinate to its corresponding finger channel.

The underlying rationale is that even if the routing algorithm is imperfect, it introduces a beneficial spatial bias. For instance, the thumb and the pinky are highly unlikely to be routed to the same channel. This structural separation simplifies the feature space, allowing the Random Forest classifier to more easily distinguish between the different classes.

2 LabVIEW

2.1 Introduction & Usage

LabVIEW was used as an interface at three main stages throughout this project:

2.1.1 Acquisition

It was used to create a baseline dataset containing the necessary information from the LMC. Specifically, the analyzed and stored data include:

- **Tip-Position(Stabilized).XYZ**: the 3D coordinates of the fingertips.
- **Palm-Position(Stabilized).XYZ**: the 3D coordinates of the palm.
- **Direction.XYZ**: 3D vector of the hand direction.
- **Timestamp**.

2.1.2 Data transformation

- **Blind channel & Routing**: These data were converted into spike trains using the Izhikevich neuron model. Specifically, all the different neuron types, obtained by varying the parameters a, b, c, d, were considered and studied using various approaches.
  First, the Euclidean distance between the *Tip* and the *Palm* was calculated, and the resulting value was then divided by a constant(scaled by 10).
  After this initial transformation, this new value serves as the **input current** for the different Izhikevich neurons. Specifically, we utilize 5 channels, and consequently 5 parallel neurons sharing the same type within each configuration, which was varied to evaluate the classification performance of the different neuron types.
- **Blind with Direction**: We additionally implemented an 11-channel setup to utilize the *Direction* vector, as empirical tests showed it provides valuable information for determining the pose. The vector was converted into spike trains by assigning two channels to each coordinate: one for positive values and one for negative values. This preserves the sign information and we decided to keep the values normalized, unlike the scaled distance metrics. We chose this method to guarantee some spiking activity even for very small values(Normalized between $[3.8 - 11]$). Thus, out of these 6 channels, exactly 3 will fire at any given time.

2.1.3 Sender and receiver

We created two VIs with different selector, that allows to communicate between different computer with UDPs. In one of the computer we use the hardware, we send the data via UDP and on other one we do the Inference.

2.2 Explanation of the Main VIs

Multiple versions of the VIs were developed, but the most important ones are described below:

1. **hand_dataset_generation_v002**: provided directly by Professor Oddo. It was used to create a baseline dataset containing all the essential values required for the neuromorphic conversion. This approach decoupled the initial data acquisition from the spike train generation, allowing us to perform the measurements just once.

2. **Multi_Izzie_csv**: This VI offers three possible choices: *Blind channel*, *Routing Node*, and *Blind with dir*. The main purpose of this VI is to convert the initial dataset into a spike train, thus serving as the core neuromorphic architecture. It was developed to simulate each neuron type from the same dataset, taking approximately 2 hours to generate the files for every neuron type. It outputs two kinds of datasets: one recording the voltage (a larger .csv file, $\approx$ 30 MB) and another logging only the spike count per second ($\approx$ 30 KB).

3. **Final_sender**: Similarly, this VI let the user to decide between which features to send via UDP. The architecture is straightforward, featuring two main *while* loops:

   (a) Acquires data directly from the LMC (there are no block related to the velocity of the loop).

   (b) Send data via UDP an array/scalar , vary depending on the selected features and converts this data into a spike trains and send the spikes count via UDP .

4. **Final_receiver**: This has many selector that allow you to decide which model you want to use for the inference, so the selector just switch between the different .pkl files. It simply receives the data array/scalar and visualizes the class predicted by the Random Forest algorithm.

5. **Neuron_model_selector**: This is a SubVI provides the ability to adjust the parameters a, b, c, d to change the neuron type .

6. **Subizzie**: This is the SubVI where the Izhikevich neuron model is implemented. We added a variable(counter of the spike) to suit our specific needs.

3 Python

This section presents the Python-based computational and machine learning framework designed for the project. While LabVIEW handles the hardware interface, real-time telemetry acquisition, and coordinate preprocessing, Python is utilized as the core analytical engine to perform offline model training, hyperparameter optimization, and real-time classification inference.

3.1 Inference

To deploy the trained classifiers in a real-time environment, a dedicated Python bridge script (`labview_inference.py`) is integrated into LabVIEW's Python Node. This node executes within the real-time block diagram loop, passing live spiking count average to Python and receiving instant gesture predictions.

3.2 Machine Learning

The workflow was structured across two distinct Python script: the first was dedicated to implementing the machine learning algorithm on the neuromorphic data, while the second applied the same pipeline to raw distance data to enable a performance comparison.
Finally the .pkl files are saved for use in the script mentioned before (Section 3.1)

3.2.1 Model Training and Hyperparameter Optimization

For both approaches, a **Random Forest Classifier** is employed.
To systematically optimize performance, a comparative study is performed between a baseline Random Forest model with default hyperparameters and an optimized model tuned via Grid Search. The optimization process is structured as follows:

- **Cross-Validation Strategy**: 5-Fold Stratified Cross-Validation is implemented on the training set (75% of the total data) to guarantee class representation across folds and avoid overfitting.
- **Grid Search Space**: The search grid evaluates 36 distinct hyperparameter combinations:
  - Number of trees ($n\_estimators \in \{50, 100, 200\}$);
  - Maximum tree depth ($max\_depth \in \{None, 10, 20\}$);
  - Minimum samples required to split an internal node ($min\_samples\_split \in \{2, 5\}$);
  - Minimum samples required to be at a leaf node ($min\_samples\_leaf \in \{1, 2\}$).
- **Model Validation**: The optimized hyperparameters are selected based on cross-validation accuracy. The chosen configurations are then evaluated on an independent test set (25% of the total data) and compared with the default baseline models.

4 Results

In this section, we show and analyze the results from our experiments with the Random Forest classifiers.

4.1 Neuromorphic Hand Gesture Classification

Neuron Type	With Routing		Without Routing	
	Default	Optimized	Default	Optimized
Fast	0.9735 ± 0.0138	**0.9754** ± 0.0113	0.8447 ± 0.0172	0.8561 ± 0.0233
Chattering	0.9697 ± 0.0150	0.9716 ± 0.0168	0.8580 ± 0.0176	0.8637 ± 0.0190
Low-Threshold	0.9678 ± 0.0127	0.9716 ± 0.0146	0.8371 ± 0.0308	0.8694 ± 0.0273
Regular	0.9641 ± 0.0138	0.9716 ± 0.0133	0.8580 ± 0.0164	0.8713 ± 0.0160
Thalamo2	0.9679 ± 0.0153	0.9716 ± 0.0133	0.8561 ± 0.0133	0.8713 ± 0.0160
Resonator	0.9697 ± 0.0109	0.9735 ± 0.0110	0.8656 ± 0.0246	**0.8845** ± 0.0067
IntrinsicallyB	0.9640 ± 0.0138	0.9678 ± 0.0128	0.8391 ± 0.0236	0.8542 ± 0.0207
Thalamo1	0.9584 ± 0.0140	0.9640 ± 0.0109	0.8542 ± 0.0107	0.8675 ± 0.0099

Table 1: Accuracy comparison of the Random Forest classifier before and after Grid Search

Table 4.1 shows the full performance comparison after tuning the parameters with Grid Search. We compare four different setups: with routing (Routed), without routing (Non-Routed), using only the sum of features (Sum Feature), and adding the hand direction (BlindDir).

Neuron Type	With Routing	Without Routing	Sum Feature Only	Direction-Augmented
Fast	**0.9754** ± 0.0113	0.8561 ± 0.0233	0.8030 ± 0.0078	**0.9849** ± 0.0096
Chattering	0.9716 ± 0.0168	0.8637 ± 0.0190	0.8202 ± 0.0295	0.9811 ± 0.0134
Low-Threshold	0.9716 ± 0.0146	0.8694 ± 0.0273	0.8088 ± 0.0229	0.9792 ± 0.0125
Regular	0.9716 ± 0.0133	0.8713 ± 0.0160	0.8107 ± 0.0174	0.9810 ± 0.0104
Resonator	0.9735 ± 0.0110	**0.8845** ± 0.0067	**0.8352** ± 0.0290	0.9773 ± 0.0153
Thalamo2	0.9716 ± 0.0133	0.8713 ± 0.0160	0.8107 ± 0.0174	0.9792 ± 0.0125
IntrinsicallyB	0.9678 ± 0.0128	0.8542 ± 0.0207	0.7785 ± 0.0167	0.9792 ± 0.0162
Thalamo1	0.9640 ± 0.0109	0.8675 ± 0.0099	0.7898 ± 0.0165	0.9848 ± 0.0097

Table 2: Model Accuracy Comparison.

- **Routing gives a huge boost:** Using spatial routing greatly increases accuracy for all neuron models. The improvements (*Delta*) range from +8.90% (for the *Resonator* model) to +11.36% (for the *Intrinsically Bursting* model).
- **Best setup:** When spatial routing is turned on, the **Fast** neuron model gets the highest accuracy at **97.54% ± 1.13%**.
- **Drop in accuracy without routing:** Without spatial routing, accuracy drops sharply, falling into a lower range between 85.42% and 88.45%.
- **Why space matters (Sum Feature, BlindDir):** Reduce the spatial features into a single sum hurts performance a lot. This proves that spatial details are key to recognizing gestures correctly. On the other hand, adding more details with the hand direction leads to much better results.

Looking at the accuracy for each gesture in Table 4.1, we can see a big difference in performance when we do not use spatial routing:

Neuron Type	Open Palm (G1)	Fist (G2)	Thumb Only (G3)	Pinky Only (G4)	Horns (G5)	Thumb+Pinky (G6)
Fast	0.9885	0.9535	0.6444	0.6667	0.9765	**0.9655**
Chattering	0.9885	0.9767	0.7222	0.6022	**0.9882**	0.9540
Low-Threshold	0.9885	**0.9884**	0.7111	0.6129	0.9765	0.9425
Regular	0.9885	**0.9884**	0.7667	0.5914	0.9765	0.9425
Thalamo2	0.9885	**0.9884**	0.7667	0.6022	0.9765	0.9425
Resonator	0.9885	0.9651	**0.8667**	0.5806	0.9765	0.9425
IntrinsicallyB	0.9885	**0.9884**	0.6111	0.6022	0.9765	**0.9655**
Thalamo1	0.9885	**0.9535**	0.7111	**0.6774**	0.9765	0.9195

Table 3: Class-wise classification accuracy per gesture class without routing.

- **Gestures that work well (G1, G2, G5, G6)**: Gestures with very clear or symmetric finger positions keep a high and steady accuracy across all neuron models.
  - *Open Palm (G1)* and *Fist (G2)* get near-perfect scores (over 98.85% and 95.35%). These gestures are opposite extremes (either all fingers open or all closed), so the system can tell them apart easily even if the hand moves around thanks to the count of active channels.
  - *Horns (G5)* and *Thumb+Pinky (G6)* also stay high ($> 97.65\%$ and $> 91.95\%$) because they have a unique, wide shape that does not get confused with other classes.
- **Bad results for single-finger gestures (G3, G4)**: On the other hand, gestures that use the same number of active channels, like *Thumb Only (G3)* (61.11% to 86.67%) and *Pinky Only (G4)* (58.06% to 67.74%), do much worse without spatial routing. The classifier cannot tell them apart just by looking at the number of channels. Plus, the distance from the top of the thumb to the pinky is almost the same, which causes the accuracy to drop severely.

Neuron Model	Accuracy (5-Fold CV)
Fast	0.9849 ± 0.0096
Thalamo1	0.9848 ± 0.0097
Chattering	0.9811 ± 0.0134
Regular	0.9810 ± 0.0104
IntrinsicallyB	0.9792 ± 0.0162
Low-Threshold	0.9792 ± 0.0125
Thalamo2	0.9792 ± 0.0125
Resonator	0.9773 ± 0.0153

Table 4: Classification Accuracy for BlindDir Neuromorphic Models (11 Channels: Raw Coordinates + Palm Direction)

4.2 Distance-Based Classification

As previously discussed, this method utilizes continuous data (distance and direction), thereby retaining the maximum possible amount of information:
Key findings from the evaluation of the hand geometry model include:

- **Impact of Palm Orientation:** Utilizing finger distances alone yields a highly optimized accuracy of 99.32%. However, when palm direction coordinates are incorporated, the model achieves a near-perfect accuracy of **99.99%**. Both configurations were compared, and ultimately, both demonstrated exceptional, near-flawless results.

Note: It is important to emphasize that we did not have the opportunity to evaluate these two specific methods in a live environment. Consequently, while they exhibit outstanding performance in offline evaluations, these results currently remain theoretical.

Configuration	**Accuracy**
Distances Only (5 feat.)	0.9932
Distances + Direction (8 feat.)	**0.9999**

Table 5: Classification results for the 1-second temporally smoothed hand geometry models.

5 Temporal Window Optimization

In rate-coded neuromorphic pipelines, the temporal window (T_{window}) used to integrate event-driven spikes is a key hyperparameter. It directly determines the trade-off between two conflicting system properties:

1. **Statistical accuracy**: Longer integration windows accumulate more spikes, reducing noise in rate estimation and producing a more stable feature representation.
2. **Response latency**: Shorter windows allow faster updates, reducing the delay between the user's movement and the system output. This is essential for real-time interactive applications.

To analyze this trade-off, we perform a case study using the **Resonator** model (unrouted, 5 channels). Classification accuracy is evaluated across ten temporal window sizes, ranging from 10 ms to 2,000 ms.

5.1 Experimental Results

The table below reports the classification accuracy for each temporal window.

Window Size (T_{window})	**Accuracy**
10 ms	0.7150
100 ms	0.8650
250 ms	0.8710
500 ms	0.8760
750 ms	0.8640
1,000 ms	0.8840
1,250 ms	**0.8860**
1,500 ms	0.8780
1,750 ms	0.8770
2,000 ms	0.8710

Table 6: Classification accuracy of the Blind Resonator model across different temporal windows

5.2 Accuracy vs. Latency Trade-off Analysis

The observed accuracy trend highlights different regimes of spike accumulation behavior:

1. **Very short windows (10 ms)**: At 10 ms, accuracy drops significantly to 71.50%. At this scale, the integration window is too short to reliably capture spike statistics, and the representation is dominated by noise.
2. **Stable performance region (100 ms – 2000 ms)**: For intermediate windows, performance remains relatively stable between 86.00% and 88.60%. The best result is obtained at 1,250 ms (88.60%). The small variations suggest that increasing the window beyond a certain point does not significantly improve class separability.

Based on these results, the optimal operating point for interactive neuromorphic applications is identified at **100 ms**, as it provides a strong balance between accuracy (86.50%) and normal latency, enabling responsive real-time behavior without significantly compromising performance. In the project the window implemented is the 1000 ms, as we didn't had the opportunity to test the 100 ms time window in all ours scenario.

6 Standard vs. Neuromorphic Comparison

6.1 Information Representation and Performance Trade-offs

A key focal point of this work is the fundamental difference in how information is represented between the standard and neuromorphic approaches. This distinction introduces a clear trade-off concerning accuracy, computational efficiency, and energy/data costs:

1. **Data representation and sparsity**: The standard approach utilizes high-precision floating-point values (32- or 64-bit) to represent spatial distances and orientations directly. Consequently, complete numerical coordinates must be continuously transmitted, even in the absence of significant motion.

   In contrast, the neuromorphic approach represents information via sparse spike events. Data is encoded using rate coding (i.e., the number of spikes within a specific time window); thus, rather than transmitting exact spatial coordinates, only dynamic movement activity is encoded.

2. **Data reduction**: In the neuromorphic paradigm, only spike counts are transmitted as integer values. While 32-bit integers are typically used, 8- or 16-bit integers would also suffice without loss of functionality.

   "Conversely, the standard approach relies on a continuous stream of 32- or 64-bit floating-point values, resulting in significantly higher data transmission requirements. Notably, comparing the data volume of the standard and neuromorphic datasets demonstrates that we achieved a substantial reduction in the overall dataset size."

3. **Performance vs. efficiency trade-off**: Remarkably, this drastic data reduction does not significantly degrade classification performance.

   The standard geometric approach, enhanced with palm direction information, achieves an accuracy of 99.99%.

   The neuromorphic `BlindDir` model (employing the *Fast* neuron) reaches 98.67% accuracy.

   The performance gap is minimal (approximately 1.33%), demonstrating that near-optimal accuracy can be sustained at a fraction of the data cost.

4. **Routed vs. BlindDir**: Comparing the two primary neuromorphic configurations highlights an additional trade-off:

   - **Routed**: Utilizes only 5 spike channels, which minimizes transmitted data but necessitates real-time spatial transformations at the edge for routing.
   - **BlindDir**: Utilizes 11 channels, which increases the data payload but entirely bypasses spatial routing computations.

   Overall, the **Routed** configuration transmits less than half the data of `BlindDir` while preserving highly comparable accuracy (Fast: 97.54% vs. 98.67%; Chattering: 97.16% vs. 98.11%).

6.2 Real-time UDP Implementation: Theory vs. Practice

A crucial aspect of this work is the transition from offline evaluation to a real-time implementation utilizing UDP communication. The objective is to verify whether the performance

observed in controlled experiments holds true under live operating conditions.

The results reveal distinct behavioral differences depending on the chosen neuromorphic encoding strategy:

- **Blind approach (5 channels)**: Performs reliably in real-time. Although it yields slightly lower accuracy in offline experiments compared to alternative methods, it exhibits greater robustness during live operation.
- **Routed approach**: Achieves higher accuracy in offline tests but proves more sensitive in real-time applications. Its performance relies heavily on the precise positioning of the hand relative to the Leap Motion sensor.
- **BlindDir approach**: Emerges as the optimal configuration in the real-time scenario. It is highly reliable and successfully resolves the common misclassification between the pinky and the thumb.

7 Conclusions and Future Work

In this work, we investigated the impact of different neuromorphic encoding configurations and biological neuron models for gesture classification, highlighting the trade-off between traditional frame-based geometric representations and neuromorphic spike-based coding. The main findings can be summarized as follows:

1. **Importance of spatial representation**: Reducing spatial information into a single dimension (*Spike Sum*) significantly degrades classification performance. In contrast, preserving spatial structure through routing or by including direction information (*BlindDir*) restores high accuracy, with the latter achieving the best cross-validation performance (98.67%).

2. **Trade-off between information and efficiency**: While the traditional geometric approach achieves up to 100% accuracy, it relies on high-precision floating-point representations. Neuromorphic approaches, particularly the `Routed` or the `BlidDir` configuration, offer a strong trade-off, slightly reducing accuracy in exchange for sparse, event-driven binary representations and improved efficiency.

7.1 Future Work

Based on the results of this study, several promising directions can be identified for future research:

1. **More robust routing mechanism**: Develop a more robust routing strategy to reduce sensitivity to hand posture and eliminate dependence on fixed thresholds.

2. **Dataset expansion**: Extend the dataset to evaluate generalization across a larger and more diverse set of participants. In addition, include more complex gesture classes, particularly challenging ones such as **G3** and **G4**, which are harder to discriminate.